

SplitMate

Roommate Expense Sharing & Rent Management

A full-stack SaaS web app built for the Indian market

Tracks shared expenses, balances, and settlements between roommates

The Problem

Roommates sharing expenses face:

- No clear record of who paid what
- Manual calculations prone to errors
- Float precision bugs in money calculations
- No single source of truth for balances
- Confusion when someone overpays or partially settles

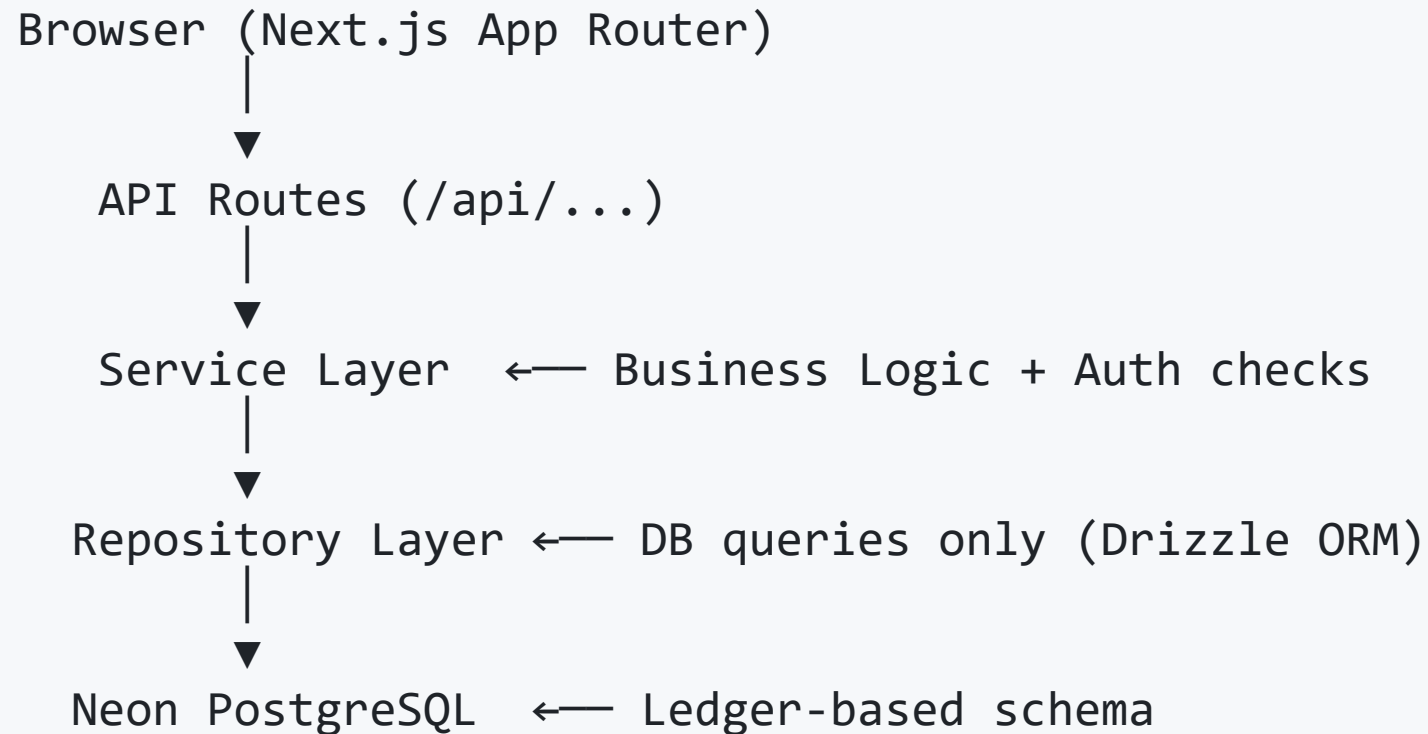
The Solution — SplitMate

- Add shared expenses with **automatic equal split**
- Track **who owes whom** in real-time
- Record **settlements** between members
- Handle **overpayments** via a credit system
- Clean **activity feed** (bank statement style)
- Invite roommates via **unique room codes**

Tech Stack

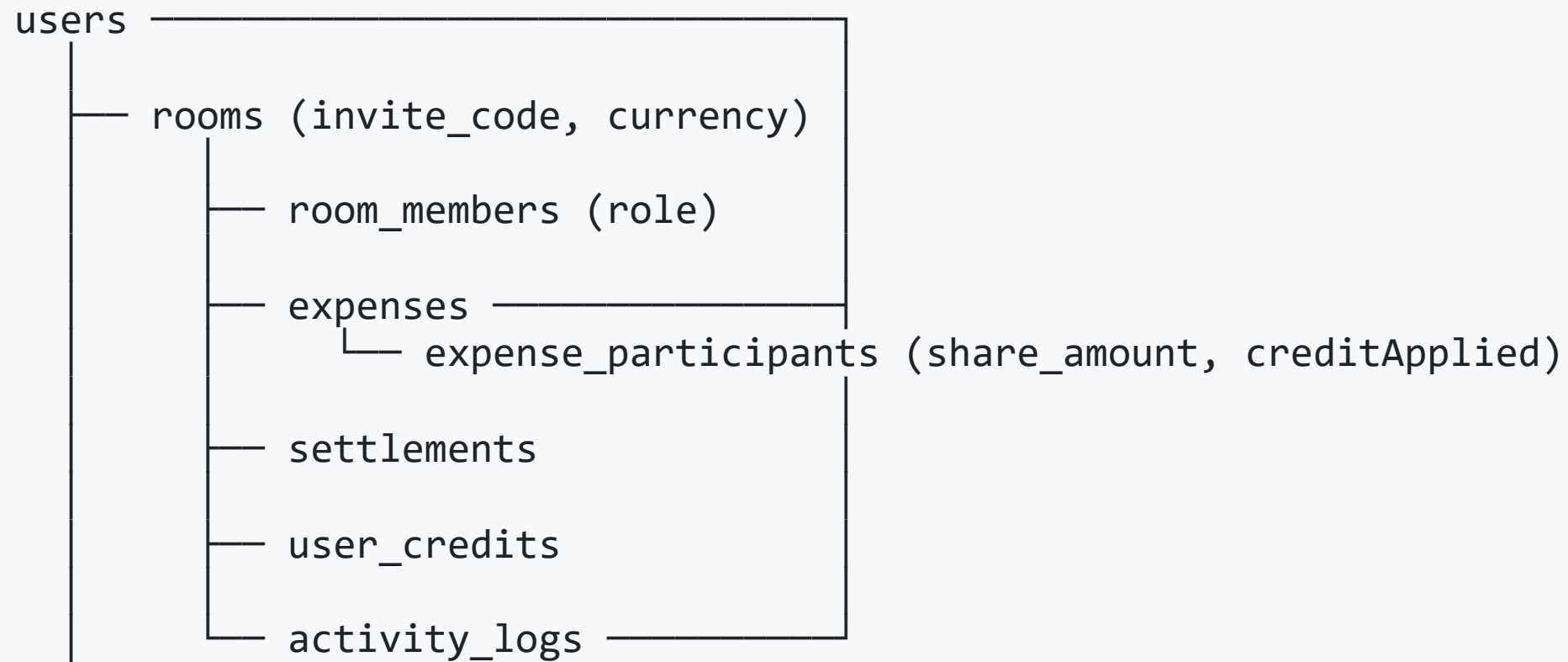
Layer	Technology	Why
Framework	Next.js 15 (App Router)	Full-stack React, SSR, API routes
Language	TypeScript (strict)	Type safety across entire codebase
Database	Neon PostgreSQL	Serverless, scalable, cost-effective
ORM	Drizzle ORM	Type-safe queries, lightweight
Auth	Clerk	OAuth, user sync via webhooks
Validation	Zod	API boundary input validation
Styling	Tailwind + Shadcn UI	Fast, consistent UI components
Deployment	Vercel	Auto-deploy from GitHub

Architecture Overview



- **Schema validation** at API boundary (Zod)
- **Auth checks** in service layer (not just middleware)
- **Every mutation** logged to `activity_logs`

Database Schema



Key Design Decisions

Decision	Reason
Integers for money (paise)	Avoid float precision bugs (₹15,000 = 1,500,000)
Ledger-based balances	Never store computed balance — always derive from raw data
Soft deletes	Audit trail integrity, no orphaned records
UUIDs everywhere	No sequential ID exposure
Clerk webhooks	Single source of truth for user identity
Zod at API boundary	No raw <code>req.body</code> usage anywhere

Core Features

1. Room Management

- Create rooms with unique 8-character invite codes
- Role-based access: **Admin** vs **Member**
- Admin can remove members (with balance warnings)

2. Expense Tracking

- Add expenses with **equal split** among members
- Categories: Rent, Groceries, Utilities, WiFi, Other
- Soft delete with full audit trail

Core Features (continued)

3. Balance Calculation (Ledger-Based)

```
net_balance =  
    (total_owed_to_you - settlements_received - virtual_credit_receipts)  
    - (total_you_owe - settlements_paid - virtual_credit_payments)  
  
positive = others owe you  
negative = you owe others
```

Balances are **always derived**, never stored — prevents inconsistency bugs

Core Features (continued)

4. Settlement System

- Record payments between members
- Auto-detects **overpayments** → creates credit record
- **Pairwise balance view** — shows exactly who owes whom

5. Credit System

- Overpayment → `user_credits` record created
- Members can **apply credit** to future expense shares
- Two paths: **Instant** (same payer) or **Proposal** (third party)
- Credit is exhausted when fully applied/settled

Activity Tab (Bank Statement Style)

3 Jun 2026

groceries	OTHER	₹350	Paid by Charan	3 Jun
└─	Charan Sai	₹116.66	Paid	
└─	Pavan	₹116.66	✓ Settled	
└─	Aiverse	₹116.68	✓ Credit settled	

 All Settlements [3]

- Per-participant **settlement status** (Pending / Settled / Credit settled)
- Date grouping (Today / Yesterday / date)

API Routes

POST	/api/rooms	Create room
GET	/api/rooms	List my rooms
POST	/api/rooms/join	Join via invite code
DELETE	/api/rooms/:id/leave	Leave room
POST	/api/rooms/:id/expenses	Add expense
DELETE	/api/rooms/:id/expenses/:eid	Soft delete expense
POST	/api/rooms/:id/settlements	Record settlement
GET	/api/rooms/:id/balances	Computed balances
GET	/api/rooms/:id/credits	Available credits
POST	/api/rooms/:id/credits	Apply credit
POST	/api/webhooks/clerk	Sync Clerk user to DB

Challenges Solved

- **Credit system** — Overpayment tracking across 3+ members with instant and proposal paths
- **Balance accuracy** — Ledger approach eliminates inconsistency bugs seen in computed-balance systems
- **Activity ordering** — Used `createdAt` (not `expense_date`) to match real-world event sequence
- **Proposal attribution** — Distinguishing third-party settlements from direct payments in balance calculation
- **Auth bootstrapping** — Auto-upsert user if Clerk webhook hasn't fired yet

Deployment

- **Live at:** splitmate.co.in
- **Hosting:** Vercel (auto-deploy from GitHub `main`)
- **Database:** Neon PostgreSQL (serverless, auto-scales)
- **Auth:** Clerk (handles OAuth, sessions, webhooks)
- **Domain:** Custom domain with SSL

Future Roadmap (Schema-ready)

- Uneven splits (percentage / exact / shares)
- UPI payment integration
- Recurring rent automation
- OCR bill scanning
- Real-time updates (WebSockets)
- Mobile app API
- Subscription plans

Thank You

SplitMate — Built for roommates, designed like a bank

Live: splitmate.co.in

Stack: Next.js · TypeScript · Neon · Drizzle · Clerk · Vercel

